KUCHENNY SKLEP INTERNETOWY

Bazy Danych
Projekt

Kierunek: Informatyka
Grupa laboratoryjna: 4

Autorzy: Gabriela Kłos (158178), Paweł Lelakowski (158188), Gabriel Pilch (158211), Piotr Niepsuj (158200)

1. Opis zadania 2
   1.1. Kto jest kim? (`users`, `addresses`) 2
   1.2. Co sprzedajemy? (`products`, `categories`, `inventory`) 2
   1.3. Jak kupują? (`orders`, `order_items`, `payments`, `shipments`) 2
   1.4. Jak to wszystko łączymy? 2
2. Relacje w bazie danych 3
3. Diagram bazy danych 4
4. Słownik danych 6
      4.0.1. Tabela `users` 6
      4.0.2. Tabela `addresses` 6
      4.0.3. Tabela `categories` 7
      4.0.4. Tabela `products` 7
      4.0.5. Tabela `product_categories` 7
      4.0.6. Tabela `inventory` 8
      4.0.7. Tabela `orders` 8
      4.0.8. Tabela `order_items` 9
      4.0.9. Tabela `payments` 9
      4.0.10. Tabela `shipments` 10
      4.0.11. Widok `v_product_stock_summary` 10
5. Uzasadnienie normalizacji 11

1. Opis zadania

Baza danych **Kuchenny Sklep Internetowy** została zaprojektowana w celu zarządzania wszystkimi kluczowymi aspektami funkcjonowania sklepu internetowego zajmującego się sprzedażą sprzętu i akcesoriów kuchennych. Obejmuje dane osobowe klientów, książkę adresową, katalog produktów wraz z kategoriami, stany magazynowe, zamówienia, płatności oraz przesyłki.

Baza danych posiada następujące encje: `users`, `addresses`, `categories`, `products`, `product_categories`, `inventory`, `orders`, `order_items`, `payments`, `shipments` oraz widok raportowy `v_product_stock_summary`.

1.1. Kto jest kim? (`users`, `addresses`)

- Każdy klient ma dokładnie jedno konto w tabeli `users`, identyfikowane przez `user_id` i unikalny adres e-mail.
- Jeden użytkownik może posiadać wiele adresów zapisanych w tabeli `addresses` (relacja 1:N). Każdy adres należy do dokładnie jednego użytkownika.
- Adresy pełnią rolę zarówno adresów dostawy, jak i rozliczeniowych. Dzięki złożonym kluczom obcym `(user_id, address_id)` zamówienie może wskazywać wyłącznie adresy należące do kupującego.
- Dane uwierzytelniania są przechowywane w tabeli `users` w postaci adresu e-mail oraz zahaszowanego hasła (`password_hash`).

1.2. Co sprzedajemy? (`products`, `categories`, `inventory`)

- Katalog produktów znajduje się w tabeli `products`; każdy produkt ma unikalny kod `sku`, nazwę, markę i cenę katalogową.
- Kategorie tworzą strukturę hierarchiczną dzięki relacji zwrotnej `parent_id` w tabeli `categories`.
- Produkt może należeć do wielu kategorii, a kategoria może zawierać wiele produktów. Relację N:M realizuje tabela asocjacyjna `product_categories`.
- Każdy produkt ma dokładnie jeden rekord stanu magazynowego w tabeli `inventory` (relacja 1:1). Ilość dostępnych sztuk nie może być ujemna.

1.3. Jak kupują? (`orders`, `order_items`, `payments`, `shipments`)

- Klient może złożyć wiele zamówień. Każde zamówienie w tabeli `orders` jest powiązane z kupującym oraz adresem dostawy i adresem rozliczeniowym.
- Jedno zamówienie może zawierać wiele pozycji w tabeli `order_items`; każda pozycja wskazuje produkt, liczbę sztuk i cenę jednostkową z chwili zakupu.
- Zamówienie może mieć wiele prób płatności w tabeli `payments` (np. pierwszą nieudaną i drugą zakończoną sukcesem).
- Jedno zamówienie może zostać podzielone na wiele przesyłek w tabeli `shipments`, np. gdy produkty są wysyłane w osobnych paczkach.

1.4. Jak to wszystko łączymy?

Najważniejsze relacje można podsumować następująco:

- `users` 1:N `addresses` oraz `users` 1:N `orders`,
- `orders` 1:N `order_items`, `orders` 1:N `payments` oraz `orders` 1:N `shipments`,
- `products` 1:1 `inventory`,
- `products` N:M `categories` przez tabelę `product_categories`,
- `addresses` 1:N `orders` w dwóch rolach: adresu dostawy i adresu rozliczeniowego.

Dzięki takiemu podziałowi dane osobowe, adresowe, katalogowe i transakcyjne są od siebie oddzielone, a integralność relacji wymuszają klucze obce oraz ograniczenia `CHECK`.

2. Relacje w bazie danych

W projekcie występują następujące relacje między encjami:

Relacje 1:N (jeden do wielu):

- `users` (1) → (N) `addresses` - użytkownik może zapisać wiele adresów w książce adresowej (`ON DELETE CASCADE`).
- `users` (1) → (N) `orders` - użytkownik może złożyć wiele zamówień (`ON DELETE RESTRICT`).
- `addresses` (1) → (N) `orders` (jako `shipping_address_id` oraz `billing_address_id`) - jeden adres może być użyty w wielu zamówieniach (`ON DELETE RESTRICT`). Klucze obce są złożone z `(user_id, address_id)`, więc adres dostawy i rozliczenia musi należeć do kupującego.
- `categories` (1) → (N) `categories` (`parent_id`) - relacja rekurencyjna (drzewo kategorii, np. *Noże i akcesoria do krojenia* → *Noże szefa kuchni*, `ON DELETE SET NULL`).
- `orders` (1) → (N) `order_items` - jedno zamówienie składa się z wielu pozycji (`ON DELETE CASCADE`).
- `orders` (1) → (N) `payments` - zamówienie może mieć wiele prób płatności (np. pierwsza nieudana, druga opłacona, `ON DELETE RESTRICT`).
- `orders` (1) → (N) `shipments` - zamówienie może zostać wysłane w jednej lub kilku paczkach (`ON DELETE CASCADE`).

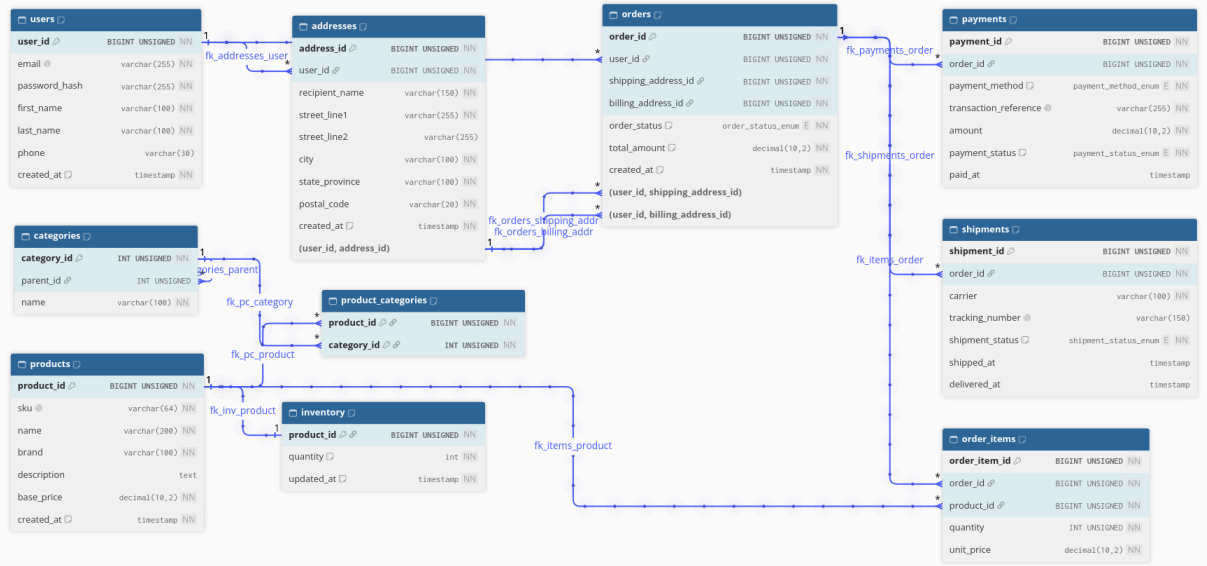
Relacje 1:1 (jeden do jednego):

- `products` (1) → (1) `inventory` - każdy produkt ma dokładnie jeden rekord stanu magazynowego (`PRIMARY KEY` w `inventory` to `product_id`, `ON DELETE CASCADE`).

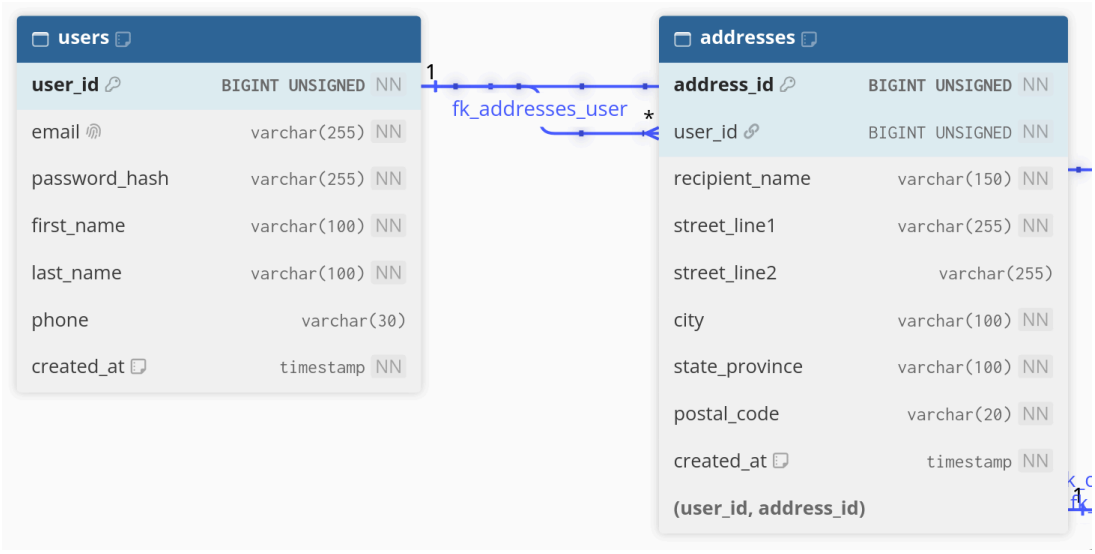
Relacje N:M (wiele do wielu):

- `products` (N) ↔ (M) `categories` poprzez tabelę asocjacyjną `product_categories` (produkt może należeć do kilku kategorii, a kategoria zawiera wiele produktów).

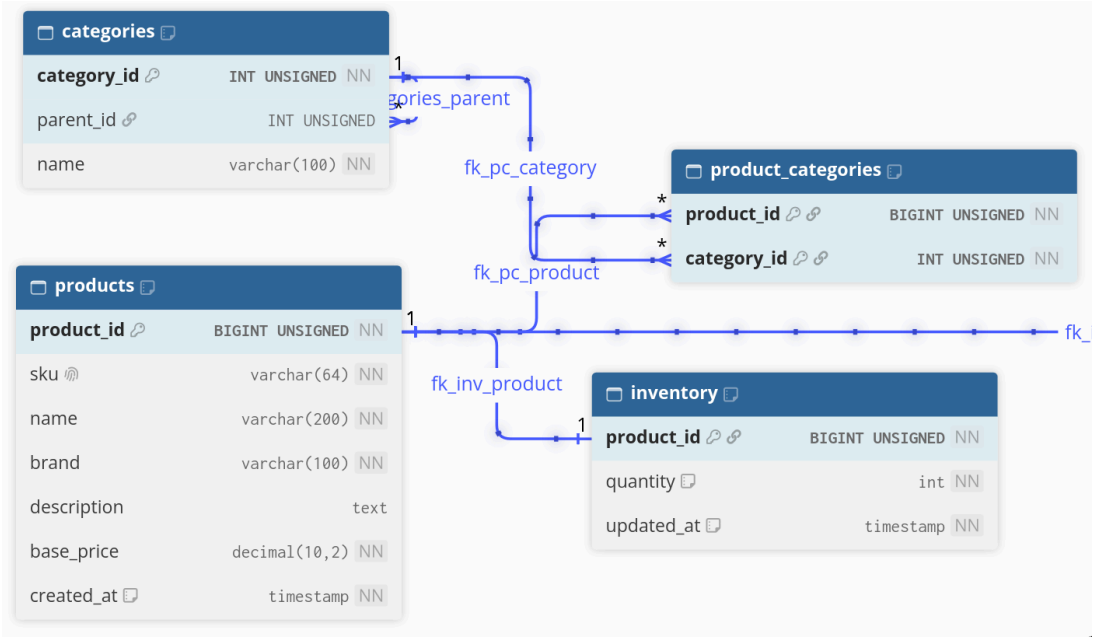
3. Diagram bazy danych



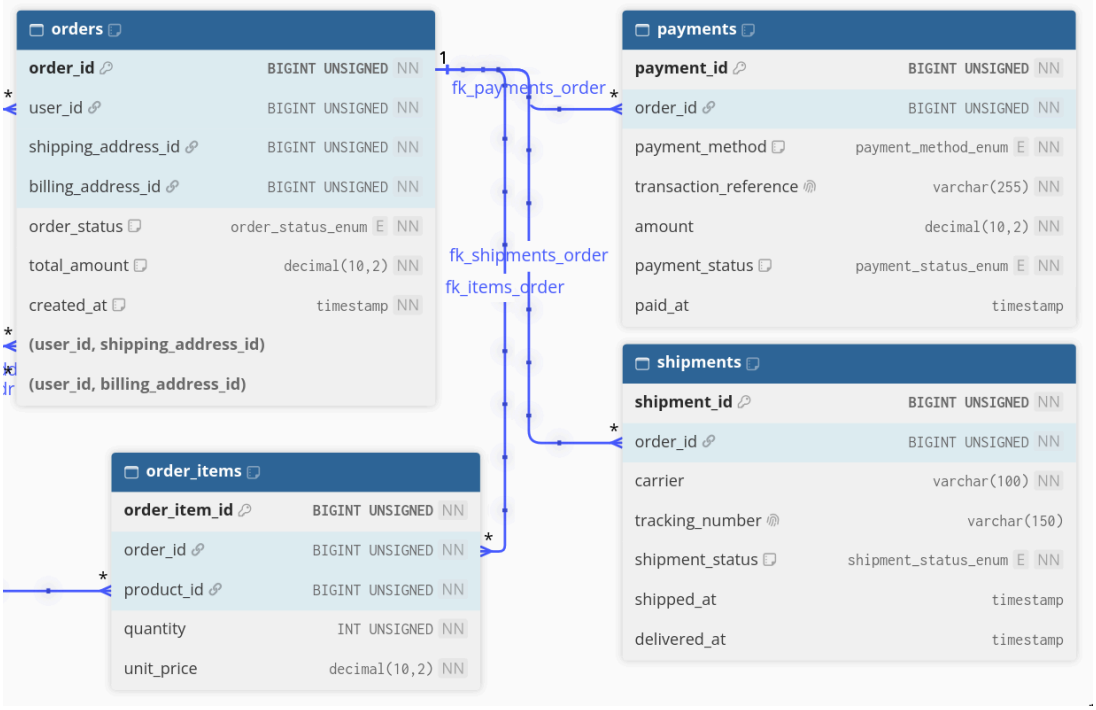
Rysunek 1: Diagram relacji bazy danych sklepu kuchennego z podświetlonymi połączeniami między tabelami.



Rysunek 2: Przybliżenie: tabele `users` i `addresses` oraz ich powiązania.



Rysunek 3: Przybliżenie: tabele `products`, `categories`, `product_categories` oraz `inventory`.



Rysunek 4: Przybliżenie: tabele `orders`, `order_items`, `payments` oraz `shipments`.

4. Słownik danych

Szczegółowe zestawienie wszystkich 10 tabel tworzących schemat bazy danych sklepu internetowego z wyposażeniem kuchennym wraz z opisem ich przeznaczenia, typami danych oraz więzami integralności.

4.0.1. Tabela `users`

Przechowuje dane kont zarejestrowanych klientów sklepu (zarówno klientów indywidualnych, jak i przedstawicieli restauracji). Stanowi główną encję uwierzytelniania oraz właściciela zamówień i książki adresowej.

Kolumna	Typ danych	Ograniczenia (Constraints)	Opis atrybutu
`user_id`	`BIGINT UNSIGNED`	`PK`, `AUTO_INCREMENT`, `NOT NULL`	Unikalny identyfikator klienta
`email`	`VARCHAR(255)`	`NOT NULL`, `UNIQUE`, `CHECK` (format e-mail, bez białych znaków)	Adres e-mail (służy jako login użytkownika)
`password_hash`	`VARCHAR(255)`	`NOT NULL`, `CHECK` (długość ≥ 20 znaków)	Zahaszowane hasło użytkownika (np. algorytm bcrypt)
`first_name`	`VARCHAR(100)`	`NOT NULL`, `CHECK` (niepuste)	Imię klienta
`last_name`	`VARCHAR(100)`	`NOT NULL`, `CHECK` (niepuste)	Nazwisko klienta
`phone`	`VARCHAR(30)`	`NULL`, `CHECK` (dozwolone znaki, 7–15 cyfr)	Numer telefonu kontaktowego dla kuriera
`created_at`	`TIMESTAMP`	`NOT NULL`, `DEFAULT CURRENT_TIMESTAMP`	Data i czas rejestracji konta

4.0.2. Tabela `addresses`

Książka adresowa użytkowników. Przechowuje adresy dostaw (np. dom, lokal gastronomiczny) oraz dane adresowe wykorzystywane do wystawiania faktur (adresy rozliczeniowe).

Kolumna	Typ danych	Ograniczenia (Constraints)	Opis atrybutu
`address_id`	`BIGINT UNSIGNED`	`PK`, `AUTO_INCREMENT`, `NOT NULL`	Unikalny identyfikator adresu
`user_id`	`BIGINT UNSIGNED`	`FK (users.user_id)`, `NOT NULL`	Użytkownik, do którego należy dany adres
`recipient_name`	`VARCHAR(150)`	`NOT NULL`, `CHECK` (niepuste)	Imię i nazwisko odbiorcy lub nazwa firmy/restauracji
`street_line1`	`VARCHAR(255)`	`NOT NULL`, `CHECK` (niepuste)	Ulica wraz z numerem budynku i lokalu
`street_line2`	`VARCHAR(255)`	`NULL`, `CHECK` (NULL lub niepuste)	Dodatkowe wskazówki adresowe (np. piętro, klatka)
`city`	`VARCHAR(100)`	`NOT NULL`, `CHECK` (niepuste)	Miejscowość
`state_province`	`VARCHAR(100)`	`NOT NULL`, `CHECK` (niepuste)	Województwo lub region administracyjny
`postal_code`	`VARCHAR(20)`	`NOT NULL`, `CHECK` (format `NN-NNN`)	Kod pocztowy
`created_at`	`TIMESTAMP`	`NOT NULL`, `DEFAULT CURRENT_TIMESTAMP`	Data dodania adresu do książki adresowej

4.0.3. Tabela `categories`

Słownik kategorii asortymentu kuchennego obsługujący strukturę drzewiastą (hierarchiczną) dzięki relacji zwrotnej `parent_id`. Pozwala dzielić główne działy (np. *Noże kuchenne*) na węższe podkategorie (np. *Noże Santoku*).

Kolumna	Typ danych	Ograniczenia (Constraints)	Opis atrybutu
`category_id`	`INT UNSIGNED`	`PK`, `AUTO_INCREMENT`, `NOT NULL`	Unikalny identyfikator kategorii
`parent_id`	`INT UNSIGNED`	`FK (categories.category_id)`, `NULL`	Wskazanie na kategorię nadrzędną (`NULL` dla głównych)
`name`	`VARCHAR(100)`	`NOT NULL`, `CHECK` (niepuste)	Nazwa kategorii wyświetlana w menu sklepu

4.0.4. Tabela `products`

Główny katalog produktów oferowanych przez sklep (noże szefa kuchni, zastawa porcelanowa, patelnie żeliwne, garnki i akcesoria). Przechowuje parametry fizyczne towaru oraz jego bieżącą cenę katalogową.

Kolumna	Typ danych	Ograniczenia (Constraints)	Opis atrybutu
`product_id`	`BIGINT UNSIGNED`	`PK`, `AUTO_INCREMENT`, `NOT NULL`	Unikalny identyfikator artykułu kuchennego
`sku`	`VARCHAR(64)`	`NOT NULL`, `UNIQUE`, `CHECK` (litery, cyfry, _, -)	Unikalny kod magazynowy produktu (Stock Keeping Unit)
`name`	`VARCHAR(200)`	`NOT NULL`, `CHECK` (niepuste)	Pełna nazwa handlowa produktu
`brand`	`VARCHAR(100)`	`NOT NULL`, `CHECK` (niepuste)	Marka / producent (np. *Zwilling*, *Gerlach*, *Le Creuset*)
`description`	`TEXT`	`NULL`, `CHECK` (NULL lub niepuste)	Szczegółowy opis zastosowania i parametrów technicznych
`base_price`	`DECIMAL(10,2)`	`NOT NULL`, `CHECK (base_price > 0)`	Aktualna cena katalogowa brutto (w PLN)
`created_at`	`TIMESTAMP`	`NOT NULL`, `DEFAULT CURRENT_TIMESTAMP`	Data wprowadzenia produktu do oferty

4.0.5. Tabela `product_categories`

Tabela asocjacyjna (łącząca) realizująca relację wiele-do-wielu (N:M) pomiędzy produktami a kategoriami. Umożliwia przypisanie jednego artykułu kuchennego do kilku kategorii jednocześnie.

Kolumna	Typ danych	Ograniczenia (Constraints)	Opis atrybutu
`product_id`	`BIGINT UNSIGNED`	`PK`, `FK (products.product_id)`, `ON DELETE CASCADE`	Identyfikator przypisywanego produktu
`category_id`	`INT UNSIGNED`	`PK`, `FK (categories.category_id)`, `ON DELETE CASCADE`	Identyfikator kategorii, do której należy produkt

4.0.6. Tabela `inventory`

Przechowuje aktualny stan magazynowy każdego produktu. Każdy produkt ma dokładnie jeden rekord stanu (`PRIMARY KEY` to `product_id`). Zapobiega sprzedaży towaru poniżej zera dzięki ograniczeniu `CHECK (quantity >= 0)`.

Kolumna	Typ danych	Ograniczenia (Constraints)	Opis atrybutu
`product_id`	`BIGINT UNSIGNED`	`PK`, `FK (products.product_id)`, `NOT NULL`, `ON DELETE CASCADE`	Identyfikator składowanego produktu
`quantity`	`INT`	`NOT NULL`, `DEFAULT 0`, `CHECK (>= 0)`	Liczba dostępnych sztuk produktu
`updated_at`	`TIMESTAMP`	`NOT NULL`, `DEFAULT CURRENT_TIMESTAMP`, `ON UPDATE CURRENT_TIMESTAMP`	Czas ostatniej zmiany stanu magazynowego

4.0.7. Tabela `orders`

Tabela nagłówków zamówień składanych przez klientów. Łączy kupującego z wybranym adresem dostawy i adresem rozliczeniowym oraz przechowuje aktualny status realizacji i łączną wartość koszyka.

Kolumna	Typ danych	Ograniczenia (Constraints)	Opis atrybutu
`order_id`	`BIGINT UNSIGNED`	`PK`, `AUTO_INCREMENT`, `NOT NULL`	Unikalny numer zamówienia
`user_id`	`BIGINT UNSIGNED`	`FK (users.user_id)`, `NOT NULL`	Klient składający zamówienie
`shipping_address_id`	`BIGINT UNSIGNED`	`FK (user_id, shipping_address_id) → addresses (user_id, address_id)`, `NOT NULL`	Adres fizycznej dostawy zamówienia (musi należeć do kupującego)
`billing_address_id`	`BIGINT UNSIGNED`	`FK (user_id, billing_address_id) → addresses (user_id, address_id)`, `NOT NULL`	Adres rozliczeniowy (do faktury, musi należeć do kupującego)
`order_status`	`ENUM(...)`	`NOT NULL`, `DEFAULT 'pending'`, `CHECK` (dozwolone wartości)	Status zamówienia (`pending`, `paid`, `processing`, `shipped`, `delivered`, `cancelled`)
`total_amount`	`DECIMAL(10,2)`	`NOT NULL`, `DEFAULT 0.00`, `CHECK (>= 0)`	Łączna wartość brutto wszystkich pozycji zamówienia
`created_at`	`TIMESTAMP`	`NOT NULL`, `DEFAULT CURRENT_TIMESTAMP`	Data i czas złożenia zamówienia

4.0.8. Tabela `order_items`

Szczegółowy wykaz pozycji wchodzących w skład zamówienia. Rejestruje zamawianą ilość oraz historyczną cenę jednostkową obowiązującą w momencie zakupu.

Kolumna	Typ danych	Ograniczenia (Constraints)	Opis atrybutu
`order_item_id`	`BIGINT UNSIGNED`	`PK`, `AUTO_INCREMENT`, `NOT NULL`	Unikalny identyfikator pozycji zamówienia
`order_id`	`BIGINT UNSIGNED`	`FK (orders.order_id)`, `NOT NULL`, `ON DELETE CASCADE`, `UNIQUE (order_id, product_id)`	Odwołanie do nadrzędnego zamówienia
`product_id`	`BIGINT UNSIGNED`	`FK (products.product_id)`, `NOT NULL`, `ON DELETE CASCADE`	Zamówiony produkt kuchenny
`quantity`	`INT UNSIGNED`	`NOT NULL`, `CHECK (quantity > 0)`	Liczba zamówionych sztuk danego artykułu
`unit_price`	`DECIMAL(10,2)`	`NOT NULL`, `CHECK (unit_price > 0)`	Cena jednostkowa brutto produktu w chwili zakupu

4.0.9. Tabela `payments`

Rejestr transakcji płatniczych powiązanych z zamówieniami. Pozwala śledzić wybraną metodę płatności, zewnętrzne numery referencyjne bramek płatniczych, nieudane próby oraz zwroty.

Kolumna	Typ danych	Ograniczenia (Constraints)	Opis atrybutu
`payment_id`	`BIGINT UNSIGNED`	`PK`, `AUTO_INCREMENT`, `NOT NULL`	Unikalny identyfikator operacji płatniczej
`order_id`	`BIGINT UNSIGNED`	`FK (orders.order_id)`, `NOT NULL`	Identyfikator opłacanego zamówienia
`payment_method`	`ENUM(...)`	`NOT NULL`, `CHECK` (dozwolone wartości)	Kanał płatności (`credit_card`, `paypal`, `bank_transfer`, `stripe`, `blik`)
`transaction_reference`	`VARCHAR(255)`	`NOT NULL`, `UNIQUE`, `CHECK` (niepuste)	Unikalny identyfikator transakcji z zewnętrznego systemu płatności
`amount`	`DECIMAL(10,2)`	`NOT NULL`, `CHECK (amount > 0)`	Kwota płatności w PLN
`payment_status`	`ENUM(...)`	`NOT NULL`, `DEFAULT 'initiated'`, `CHECK` (dozwolone wartości)	Status transakcji (`initiated`, `completed`, `failed`, `refunded`)
`paid_at`	`TIMESTAMP`	`NULL`, `CHECK` (spójność ze statusem)	Data i czas potwierdzenia wpłaty (`NOT NULL` dla `completed`/`refunded`, `NULL` dla `initiated`/`failed`)

4.0.10. Tabela `shipments`

Ewidencja przesyłek kurierskich i paczkomatowych realizujących dostawę zamówienia. Jedno zamówienie może być podzielone na wiele przesyłek (np. przy wysyłce w osobnych paczkach).

Kolumna	Typ danych	Ograniczenia (Constraints)	Opis atrybutu
`shipment_id`	`BIGINT UNSIGNED`	`PK`, `AUTO_INCREMENT`, `NOT NULL`	Unikalny identyfikator przesyłki
`order_id`	`BIGINT UNSIGNED`	`FK (orders.order_id)`, `NOT NULL`	Identyfikator wysyłanego zamówienia
`carrier`	`VARCHAR(100)`	`NOT NULL`, `CHECK` (niepuste)	Nazwa przewoźnika (np. *InPost Paczkomaty*, *DPD Polska*, *DHL*)
`tracking_number`	`VARCHAR(150)`	`NULL`, `UNIQUE`, `CHECK` (NULL lub niepuste)	Unikalny numer listu przewozowego do śledzenia paczki
`shipment_status`	`ENUM(...)`	`NOT NULL`, `DEFAULT 'label_created'`, `CHECK` (dozwolone wartości)	Status logistyczny (`label_created`, `in_transit`, `out_for_delivery`, `delivered`)
`shipped_at`	`TIMESTAMP`	`NULL`, `CHECK` (spójność ze statusem)	Data i czas przekazania paczki kurierowi
`delivered_at`	`TIMESTAMP`	`NULL`, `CHECK` (spójność ze statusem i `shipped_at`)	Data i czas doręczenia przesyłki odbiorcy

4.0.11. Widok `v_product_stock_summary`

Widok raportowy łączący produkty z ich aktualnym stanem magazynowym. Dzięki zastosowaniu `LEFT JOIN` produkty bez rekordu w tabeli `inventory` również pojawiają się w wyniku (z `total_stock` równym 0).

Kolumna	Typ danych	Opis
`product_id`	`BIGINT UNSIGNED`	Identyfikator produktu
`sku`	`VARCHAR(64)`	Kod magazynowy produktu
`product_name`	`VARCHAR(200)`	Nazwa produktu
`brand`	`VARCHAR(100)`	Marka / producent
`base_price`	`DECIMAL(10,2)`	Cena katalogowa produktu
`total_stock`	`INT`	Łączny stan magazynowy (`0`, jeśli brak rekordu w `inventory`)
`total_price`	`DECIMAL(10,2)`	Wartość stanu magazynowego (`base_price * total_stock`)

5. Uzasadnienie normalizacji

Zaprojektowany schemat bazy danych spełnia wymogi co najmniej **trzeciej postaci normalnej (3NF)**:

1. **Pierwsza Postać Normalna (1NF):**
   - Wszystkie atrybuty w tabelach są **atomowe** (niepodzielne) - np. dane osobowe zostały rozbite na `first_name` i `last_name`, a dane adresowe na `street_line1`, `street_line2`, `city`, `state_province` oraz `postal_code`.
   - Brak powtarzających się grup kolumn (np. zamiast kolumn `category_1`, `category_2` w tabeli `products`, wydzielono tabelę łączącą `product_categories`).
   - Każda tabela posiada zdefiniowany klucz główny (`PRIMARY KEY`).
2. **Druga Postać Normalna (2NF):**
   - Spełnione są warunki 1NF, a każdy atrybut niebędący częścią klucza zależy **w pełni funkcjonalnie** od całego klucza głównego.
   - W tabeli `product_categories` ze złożonym kluczem głównym `(product_id, category_id)` nie występują atrybuty niekluczowe, więc nie ma możliwości częściowej zależności. W pozostałych tabelach klucz główny jest pojedynczą kolumną, a wszystkie atrybuty niekluczowe zależą od niego w całości.
3. **Trzecia Postać Normalna (3NF):**
   - Spełnione są warunki 2NF oraz **nie występują zależności przechodnie** (atrybuty niekluczowe zależą wyłącznie od klucza głównego, a nie od innych atrybutów niekluczowych).
   - Adresy dostawy i adresy rozliczeniowe zostały wydzielone do osobnej tabeli `addresses`, dzięki czemu zmiana adresu użytkownika lub przypisanie go do zamówienia nie powoduje duplikowania danych adresowych w tabeli `orders`.
   - **Uwaga projektowa dotycząca cen historycznych:** Kolumna `unit_price` w tabeli `order_items` przechowuje cenę produktu **w momencie zakupu** (np. z uwzględnieniem ówczesnego cennika lub promocji), natomiast `base_price` w tabeli `products` to bieżąca cena katalogowa. Dzięki temu późniejsza zmiana ceny noża czy garnka w katalogu nie fałszuje historycznych faktur i zamówień.